BallMotion.h

02 / Referencia de API matemática

Módulo header-only en namespace ball. No depende de Arduino, I²C, serial ni Unity. Recibe mediciones físicas y conserva una orientación; devuelve una orden de movimiento.

Fuente: include/BallMotion.h:1. Configuración compartida: include/ControllerConfig.h:1.

Convenciones y contrato

Gyro en rad/s, acelerómetro en g, dt en segundos. Las entradas se expresan en los ejes del sensor. La referencia calibrada usa X derecha, Y adelante, Z arriba (dextrógira). orientation rota sensor → referencia. Datos finitos y cuaternión unitario son precondiciones.

Tipo / miembro	Contrato
Vec(x,y,z)	Vector de tres float; constructor por defecto en cero. + y - por componente; * escala; norm() calcula magnitud euclídea.
cross(Vec a, Vec b)	Producto vectorial a × b. Se usa internamente para la corrección de inclinación.
Quat(w,x,y,z)	Identidad por defecto: (1,0,0,0). Constructor explícito por componentes.
Quat::operator*	Producto de Hamilton; composición no conmutativa.
Quat::inverse()	Devuelve conjugado; equivale a inversa solo para cuaterniones unitarios.
Quat::normalize()	Divide por la norma. Requiere norma no nula; no implementa reparación de entradas arbitrarias.
Quat::rotate(Vec)	Aplica q·(0,v)·q inverso. Mantiene magnitud si q es unitario.
Output	right y forward, cero por defecto. Orden adimensional, no posición ni aceleración.

Estado público de Motion

orientation: cuaternión persistente. fullSpeed: float mutable, inicialmente pi rad/s (180 grados/s). Debe ser mayor que la zona muerta. reset() restablece solo orientation; no modifica fullSpeed ni calcula bias.

No hay guardas generales contra NaN/Inf, cuaternión nulo o fullSpeed inválido. El firmware debe mantener estas precondiciones. No modificar orientation arbitrariamente desde un receptor.

Motion::update

02 / Algoritmo y efectos de estado

```
Output update(Vec gyro, Vec accel, float dt,
              bool movementEnabled = true);
```

1. Validación temporal

Rechaza dt ≤0 o superior a MAX_SAMPLE_GAP_SECONDS: devuelve Output cero sin modificar orientación. La función no cambia Mode ni reconecta hardware; esas responsabilidades pertenecen a main.

2. Corrección de inclinación

Copia gyro a corrected. Si la magnitud del gyro es <0.15 rad/s y la del acelerómetro está estrictamente entre 0.9 y 1.1 g, calcula la vertical esperada en ejes locales. Suma cross(accel normalizado, expected) multiplicado por la ganancia 2 por segundo a corrected. Usa constantes de ControllerConfig.h.

La condición aproxima reposo, no lo garantiza. La gravedad corrige inclinación, no observa rumbo absoluto. Cerca del reposo reduce contaminación por aceleración de movimiento; no elimina toda vibración.

3. Integración de orientación

```
speed = |corrected|
half = speed * dt / 2
dq = (cos(half), corrected * sin(half) / speed)
orientation = normalize(orientation * dq)
```

Solo integra si speed >1e-8 rad/s, para evitar división prácticamente por cero. Multiplica a la derecha porque el giro es local. Supone velocidad constante durante dt; no reconstruye muestras perdidas. Normalizar corrige error numérico, no deriva física.

4. Bloqueo de movimiento y cambio de referencia

Si movementEnabled=false, retorna cero después de actualizar orientación. Si true, calcula world=orientation.rotate(gyro). Usa gyro, no corrected: la corrección de inclinación no se agrega directamente a la orden.

5. Dirección, zona muerta y escala

```
x = world.y;  y = -world.x
m = sqrt(x*x + y*y)
if m <= deadzone: return (0,0)
gain = min(1, (m-deadzone)/(fullSpeed-deadzone))
return (x/m * gain, y/m * gain)
```

Corresponde al movimiento de la superficie superior: omega × (0,0,R), con radio absorbido en sensibilidad. La diagonal comparte el máximo circular. world.z no produce desplazamiento; al detenerse no se agrega inercia artificial.

Configuración y extensión

02 / Parámetros · restricciones · pruebas

ControllerConfig.h	Valor inicial / función
MOVEMENT_DEADZONE_DEG_PER_SEC	2 grados/s; zona muerta radial de salida. Su equivalente RAD_PER_SEC se deriva automáticamente.
TILT_MAX_GYRO_RAD_PER_SEC	0.15 rad/s; máximo para habilitar corrección de inclinación (comparación estricta).
TILT_MIN_ACCEL_G / TILT_MAX_ACCEL_G	0.9 / 1.1 g; ventana estricta de magnitud acelerométrica.
TILT_CORRECTION_GAIN_PER_SEC	2 por segundo; fuerza de corrección hacia la vertical medida.
MAX_SAMPLE_GAP_US	50000 us; pausa local máxima admitida. MAX_SAMPLE_GAP_SECONDS se deriva de este valor.

No confundir el límite local de muestreo con latencia Bluetooth. Revisar ruido de la caja, umbrales de reposo, fuerza de corrección y tiempos cuando haya hardware. fullSpeed aún es miembro de Motion, no constante centralizada.

Placeholder: onVerticalRotation(float)

Función vacía y sin llamadas. Su parámetro futuro es velocidad angular con signo sobre la vertical fija (world.z), en rad/s. No usar gyro.z local. No produce eventos, telemetría extra ni acción del juego.

Limitaciones del modelo

No estima posición, rumbo absoluto ni bias; main calcula el sesgo. Requiere pose física conocida al reiniciar para que identidad corresponda al jugador. Aceleración adicional, sesgo térmico y pérdidas de muestra pueden degradar orientación. La corrección puede modificar cómo se proyectan los giros, aunque no añade movimiento sintético.

Regresiones de host

test/motion_host.cpp comprueba reposo, ruido, signos, intensidad, diagonal, giro vertical, diez vueltas, cambios de referencia, corrección de inclinación, ausencia de corrección ficticia del rumbo, dt inválido, reset y preview con movimiento bloqueado. Ejecutar test/run-host.ps1; requiere g++ configurado. No sustituye pruebas de hardware.

Uso desde firmware: mapa de llamadas y miembros. Formato de publicación: contrato serial K2.